

ERARBEITUNGS- UND REFLEXIONSPHASE

DATA-MART-ERSTELLUNG IN

SQL – BESCHREIBUNG IMPLEMENTIERUNG

Studiengang: Software Development Expert

Mario Oppen
Matrikelnummer: UPS10725543
Tutorin: Dr.-Ing. Anna Androvitsanea
1. September 2026

1 Eingesetzte Software Tools

1.1 Auswahl des Datenbankmanagementsystems

Zur Implementierung der Buchtausch-App sollte ein Datenbankmanagementsystem (DBMS) gewählt werden, das möglichst leicht zu handhaben ist, eine breite Funktionspalette unterstützt und flexibel auf unterschiedlichen Plattformen eingesetzt werden kann. Zudem sollte das DBMS nicht nur geläufige SQL-Funktionen unterstützen, sondern dem Nutzer Möglichkeiten bieten, individuelle Funktionen und Prozeduren zu definieren, um bei Anlage und Verwaltung der Stammdaten sowie der Steuerung der Ausleihprozesse fachliche Abläufe besser zu unterstützen und den Anwender bei Ausführung von Aktionen ideal führen zu können.

Aus den möglichen Kandidaten wurde letztlich PostgreSQL als Datenbanksystem für die Umsetzung ausgewählt, da es kostenfrei als Open-Source-Lizenz verfügbar ist. Es bietet eine nahezu vollständige Unterstützung für den SQL-Standard wie auch zusätzliche Funktionen, die einen effizienten Einsatz ermöglichen. Beispielsweise lassen sich über Extensions prozedurale Sprachen oder zusätzliche Funktionen wie PostGIS integrieren (vgl. *PostgreSQL*, 2026).

1.2 Docker als Ausführungsplattform

Obwohl PostgreSQL auf einer Vielzahl von Plattformen betrieben werden kann, bietet sich für die Entwicklung und Realisierung der Buchtausch-App Docker als Plattform an. Denn die Plattform bietet mit vorgefertigten Images und der Containerstruktur die Möglichkeit, schnell und effizient mit wenigen Befehlen eine betriebsfertige Umgebung zu erstellen und diese bei Bedarf wieder zu entfernen, zu duplizieren oder zu verändern (vgl. *Docker*, 2026).

Die PostgreSQL-Umgebung der Buchtausch-App kann so durch den einzelnen, nachstehenden Befehl in einem Container gestartet werden, wobei Datenbank, Tabellen und Testdaten mit Hilfe von Initialisierungsskripten während des Startprozesses erstellt werden.

```
docker run --restart=unless-stopped --name iuLendMyBook -d -e POSTGRES_USER=iuUser -e POSTGRES_PASSWORD=* -e POSTGRES_DB=iuLendMyBook -v $PWD/init:/docker-entrypoint-initdb.d/ -v $PWD/resources:/var/lib/iu/data/test_data/ -p 5440:5432 postgres:latest
```

Der Befehl beinhaltet dabei neben dem zu verwendenden Image weitere Parameter, die während des Starts berücksichtigt werden sollen:

- **-e:** Umgebungsvariablen; setzen in diesem Fall Benutzernamen, Passwort und Datenbanknamen für die Buchtausch-App
- **-v:** legt einen virtuellen Datenträger für den Container an und verknüpft diesen mit einem Verzeichnis auf dem Host-System; beispielsweise werden im o.g. Beispiel die Skripte aus dem lokalen Ordner *init* in das Verzeichnis *docker-entrypoint-initdb.d/* kopiert, damit sie dort beim Start der Datenbank ausgeführt werden

- **-p:** gibt den Port an, unter dem die Anwendung vom Hostsystem bzw. innerhalb des Containers erreichbar ist, hier 5440 bzw. 5432

2 Implementierung der Datenbank

2.1 Änderungen zur Konzeptionsphase

Bevor die Datenbank implementiert wurde, wurde das Konzept punktuell überarbeitet. Dabei wurden die Use Cases weiter präzisiert und auf die fachlichen Anwendungsfälle zugeschnitten. U.a. wurde deutlich gemacht, dass die Bewertungen eines Ausleihvorgangs nur durch den Entleiher abgegeben werden können und durch den Moderator ausgewertet werden.

Weiterhin wurde das Lösungsdesign überarbeitet, sodass die Hinweise zur technischen Implementierung der Datenbank entfallen sind und sich das Dokument an dieser Stelle auf die fachliche Umsetzung der Lösung fokussiert.

Zuletzt wurden das Entity-Relationship-Modell (ERM) und das Data Dictionary angepasst. Dabei wurde eine neue Entität *status* eingeführt, die als Fremdschlüsselbeziehung mit den Entitäten *book_copy* und *user_account* verknüpft wurde. Diese Anpassung soll sicherstellen, dass anstelle einer einfachen Zeichenkette ein konsistenter, referenzierter Status für beide Entitäten verwendet wird.

Zudem wurden die Eindeutigkeitsmerkmale in den Entitäten `AUTHOR` und `PUBLISHER` überarbeitet, sodass in beiden Fällen Einträge eindeutig aufgrund der Kombination ihrer Attribute identifiziert werden können. Dies erleichtert in der Umsetzung und im Betrieb Such- und Einfügeoperationen.

2.2 Implementierung

Die Implementierung der Datenbank der Buchtausch-App orientiert sich vorwiegend an den Vorgaben des Data Dictionary und des ERM. Während das ER-Modell Aufschluss über die Beziehungen der Entitäten untereinander gibt, stellt das Data Dictionary für alle Relationen die anzulegenden Attribute, ihre Datentypen und die Constraints zusammen.

Im Rahmen der Implementierung werden alle modellierten Entitäten als eigene Datenbanktabelle (Relation) angelegt. Dies betrifft die zentralen Entitäten wie Buch, Buchexemplar oder Benutzer, ebenso wie die Nachschlagetabellen für Sprache, Autoren, Verlage oder die Bereitstellungsart der Buchexemplare. Für alle Tabellen werden *Surrogate Keys* (künstlicher Schlüssel) als Primärschlüssel angelegt. Die Anlage der Schlüssel erfolgt dabei einheitlich und automatisiert über die PostgreSQL-Option `GENERATED ALWAYS AS IDENTITY PRIMARY KEY`. Somit wird sichergestellt, dass je Relation eindeutige Primärschlüssel definiert werden.

Zudem werden für alle m:n-Beziehungen aus dem ER-Modell Zuordnungstabellen erstellt, damit Mehrfachzuordnungen von Attributen aufgelöst werden können. Bei diesen Zuordnungstabellen wird auf die Erstellung künstlicher Schlüssel verzichtet, da sich der Primärschlüssel aus den

Fremdschlüsseln der verknüpften Entitäten zusammensetzt. Ausnahmen bilden dabei assoziative Entitäten. Sie stellen Zuordnungstabellen dar, die eigene Attribute enthalten und als Fremdschlüssel in weiteren Tabellen referenziert werden. Für diese Relationen wird ebenfalls ein Surrogate Key festgelegt, um die Verknüpfung mit anderen Relationen zu erleichtern.

Für eine erleichterte Lesbarkeit und bessere Wiederverwendbarkeit werden alle Beziehungen (Foreign Keys), Eindeutigkeitskriterien (Unique-Constraints) und Prüfbedingungen (Check-Constraints) als einheitlicher Bedingungsblock (Constraints) formuliert. Zudem werden im Kontext der Fremdschlüsselbeziehungen referenzielle Aktionen gesetzt, die das Verhalten der Datenbank beim Löschen eines referenzierten Objekts bestimmen. Während in den meisten Fällen die Löschung eines verknüpften Datensatzes durch die Standard-Aktion `ON DELETE NO ACTION` unterbunden wird, wird die Aktion `ON DELETE CASCADE` punktuell dort gesetzt, wo durch das Löschen von Hauptobjekten verwaiste Einträge entstehen würden.

Letztlich werden auf einigen Relationen Indizes ergänzt. Dabei werden im Wesentlichen Indizes für Fremdschlüsselbeziehungen hinzugefügt, sodass `JOIN`-Abfragen effizienter ausgeführt werden können. Besonders für die Tabelle `book_loan` wird ein partieller Index auf den Status gesetzt, sodass in Ausleihe befindliche Vorgänge effizient identifiziert werden können. Dies hilft vor allem bei Suchen, um aktuell verfügbare Exemplare aufzufinden.

3 Testung

3.1 Erstellung von Testdaten

Damit die im Rahmen der Implementierung erstellten Relationen und Constraints überprüft werden können, sind im nächsten Schritt Testdatensätze für die angelegten Entitäten zu erstellen. Ziel ist es dabei fiktive, aber dennoch realistische Daten zu erzeugen. Dabei wurden für die vorliegende Implementierung drei unterschiedliche Verfahren für die Generierung verwendet:

- 1) **Verwendung öffentlicher Quellen:** Allgemeingültige Daten, die für das Befüllen der Nachschlagetabellen relevant sind, sind in der Regel im Internet verfügbar. So wurde für die Erstellung der Nachschlagetabelle `language` auf eine Quelle aus dem Auswärtigen Amt zurückgegriffen (vgl. Auswärtiges Amt, 2026). Die Daten wurden auszugsweise übernommen und in CSV-Dateien überführt.
- 2) **Einsatz künstlicher Intelligenz:** Fiktive Daten können mit Hilfe künstlicher Intelligenz generiert werden. So wurden qualifizierte Prompts entwickelt, die die benötigten Daten in einer definierten Struktur erstellt und als CSV-Datei bereitgestellt haben¹.
- 3) **Generierung verknüpfter Daten per SQL:** Bewegungsdaten, wie z.B. Ausleihvorgänge ergeben sich aus der Kombination unterschiedlicher Stammdaten der Datenbank. Valide

¹ Für die Erstellung von Benutzer- und Adressdaten, Stammdaten der Buchtitel und Zeitfenster wurden drei Prompts entwickelt. Diese werden dem GitHub-Repository beigelegt.

Daten können daher nur schwer ohne Datenbankbezug erstellt werden und wurden daher mit Hilfe zufälliger Kombinationen der Stammdaten erzeugt.

Diese generierten Daten werden während der Initialisierung der Datenbank in die Datenbank überführt, sodass alle fachlichen Stamm-, Zuordnungs- und Bewegungsdatentabellen mit mindestens zehn Beispieldatensätze befüllt werden. Die Anlage der Testdaten folgt dabei auf die Erstellung der Datenstruktur in mehreren Schritten. Zunächst werden die Stammdaten aus den erstellten CSV-Dateien mit Hilfe der Funktion `COPY` eingelesen und in die zugehörigen Zieltabellen kopiert. Im Anschluss erfolgt die Anlage komplexerer Entitäten und deren Abhängigkeiten. Da die Daten nicht direkt übernommen werden können, werden die Quelldaten zunächst in temporäre Datenbanktabellen übernommen, bevor anschließend aufeinander aufbauende Skripte Datensätze in den Relationen anlegen und abhängige Daten untereinander verknüpfen.

In der Folge werden Zuordnungen zwischen den Entitäten, wie beispielsweise Buchexemplare zu Benutzern oder Ausleihvorgänge zu Buchexemplaren über SQL-Statements realisiert. Dazu werden per `random()`-Funktion zufällig Datensätze selektiert, mit Daten angereichert und als neue Datensätze in abhängigen Tabellen angelegt. Zum Abschluss der Initialisierung werden die angelegten temporären Datenbanktabellen entfernt.

3.2 Testkonzept und Use Cases

Auf Basis der erstellten Testdaten kann die Funktionalität der Datenbank getestet werden. Zwar können die Constraints innerhalb der Relationen beim Einfügen bzw. Löschen von Datensätzen durch einfache `INSERT`- und `DELETE`-Statements getestet werden, jedoch bildet dies nur Teilmengen der Prozesse der Buchtausch-App ab. So kann beispielsweise bei Löschvorgängen innerhalb der Datenbank nicht automatisiert überprüft werden, ob die Person, die den Vorgang durchführt, dazu berechtigt ist und die korrekte Rolle innehat. Daher werden für die Tests realistische Anwendungsszenarien simuliert, indem gezielt Prozeduren und Funktionen erstellt werden, die diese Anwendungslogik kapseln. Unter anderem sind die Prozeduren zum Erstellen von Entitäten darauf ausgelegt, dass bei einer Verletzung von `UNIQUE`-Constraints keine Fehlermeldung entsteht, sondern der Fehler abgefangen wird und stattdessen der bereits vorhandene Eintrag zurückgegeben wird. Dadurch ist es möglich während des Testens mehrere Prozeduren miteinander zu verknüpfen und somit ganze Geschäftsvorfälle abzubilden. Beispielsweise erstellt die Prozedur `getOrCreateAddress` eine neue Adresse, sofern diese noch nicht vorhanden ist und verknüpft einen bestehenden Ort oder erstellt bei Bedarf einen Ort neu und verknüpft im Anschluss diesen.

In der beigefügten Präsentation werden je Entität zum einen die Skripte zur Erstellung der Relation dargestellt. Zuordnungstabellen werden gemeinsam mit den Hauptentitäten aufgeführt und beschrieben. Neben der Anlage der Entitäten enthält die Präsentation je Entität eine weitere Folie mit den Testfällen und Ergebnissen zum Anlegen, Löschen oder Bearbeiten von Datensätzen. Die Ergebnisse werden dabei als Ausgabe der Konsole dargestellt und enthalten Aufruf, sowie auch Ergebnis. Die Protokolle sind dabei auf die wesentlichen Informationen gekürzt und enthalten nur

einen Auszug aus dem Stacktrace. Für die Ausleihvorgänge und die Bewertungen werden zudem exemplarisch `SELECT`-Statements aufgeführt, um die Suche nach verfügbaren Exemplaren sowie die Auswertung abgegebener Bewertungen zu simulieren.

Quellenverzeichnis

Auswärtiges Amt. (2026, August 1). *Verzeichnis von Sprachkennungen und Sprachennamen auf Deutsch*. Verzeichnis von Sprachkennungen und Sprachennamen auf Deutsch.

<https://www.auswaertiges->

[amt.de/resource/blob/2732426/0c706d352b2aeff5862a1bdf3968044a/sprachkennungen-](https://www.auswaertiges-)
[data.pdf](https://www.auswaertiges-)

Docker. (2026, August 30). Docker Documentation - What Is Docker? <https://docs.docker.com/get-started/docker-overview/>

PostgreSQL. (2026, August 30). <https://www.postgresql.org>